CampusPrice: Build Plan for PS 2.4 Pricing and Combo Decision Support

Oct 29, 2026 · @BIT Project -PS team

Concept

CampusPrice (working name) is a responsive web co-pilot where the cafeteria owner changes a price, portion or combo, sees predicted demand and profit with a range, compares scenarios, and gets recommendations that already respect student budgets.

The owner's loop has seven steps:

1. See menu health.
2. Change an input.
3. Predict demand.
4. Compare scenarios.
5. Get a recommendation.
6. Decide: approve or override.
7. Learn from the log.

Everything below is a design to build. Only the findings from the Kaggle CSV were measured; the synthetic-data and model plans are untested.

Screens

The app has five owner screens plus one student screen. Owner screens work on laptop and tablet; the student screen is phone-first.

Screen	Owner or student does	Shows
Menu Health	Owner opens the app	Per item: price, cost, ₹ and % margin, units per day, flags (loss-maker, poor seller, star)
Demand Lab	Owner picks an item and moves price, portion, discount and day_type sliders	Response curve with a range band; units, revenue, profit and margin update live

Screen	Owner or student does	Shows
Scenario Compare	Owner saves up to 4 scenarios: Today, My scenario, AI plan, Student-first	Side-by-side units, revenue, profit, margin, students served within budget, ₹ per 100 g, risk
Combo Studio	Owner picks a main, an add-on and a discount	Take rate, add-on lift, cannibalisation, break-even take rate, incremental profit with range
Recommendations	Owner approves, edits or rejects cards	Apply, Pilot or Hold, the reason, guardrails met, test plan for pilots
Student Budget Builder	Student moves a ₹ slider	Best approved deals, "save ₹X", ₹ per gram

How the owner changes the prediction

The Demand Lab recomputes this on every slider move:

```
\text{demand} = \text{reference demand} \times (\text{price ratio})^{\text{price effect}} \times (\text{portion ratio})^{\text{portion effect}} \times \text{combo-discount effect} \times \text{day-type factor}
```

The model returns many plausible curves, not one, so the screen shows a band. A guardrail breach turns the readout red: below margin floor, above student cap, or change too large.

Worked example, illustrative, using the Kaggle-based elasticity of −2.4 for an item at ₹70, cost ₹45, 120 sold a day:

Price (₹)	Units per day	Profit per day (₹)
60	174	2,606
65	143	2,867
70	120	3,000
75	102	3,051
77 (optimum)	96	3,055

The top is flat: moving from ₹70 to ₹77 gains about 2%, while cutting to ₹60 loses about 13%. Holding the price near ₹70 for students costs little, and that is the story the frontier chart tells.

How price and combo scenarios are compared

Each scenario is a full set of price, portion and combo discount per item. The model predicts each metric with a range, and the screen shows differences against Today.

- **Discount break-even:** at a 35.7% margin, a 5% discount needs about +16% volume to break even, 10% needs +39%, and 15% needs +72%. Discounts whose predicted lift falls short are flagged.
- **Combo cannibalisation (assumed numbers):** a meal at ₹70 (cost ₹45) and a drink at ₹20 (cost ₹5), with 30% of meal buyers already adding a drink. A ₹82 combo needs about a 64% take rate to match today's profit, because students who would have bought both anyway now pay less.
- **Frontier chart:** profit against students served within budget. The owner picks a point and sees the profit given up for each extra point of coverage.

The model

The model has six layers; only the first two and the LLM layer are learned or generative, and the rest is deliberate maths and rules.

Layer	What it is	Notes
Demand response	Per-item log-linear count model with partial pooling toward category averages. Inputs: price, portion, discount, day_type, combo flags	Returns a range. Challenger: gradient boosting with a price-decreasing constraint
Combo	Take rate vs discount, add-on lift, main traffic lift, cannibalisation	Output: incremental profit per day with a range
Student	Budget tiers (tight, mid, flexible) with shares and budgets from the survey	Output: coverage = share who can afford a complete meal; ₹ per 100 g
Optimiser	Grid search: price ±15%, portion tiers, combo discounts 0, 5, 10, 15%	Maximise expected profit under guardrails and a coverage target
Confidence gate	Posterior draws give the probability that profit improves	Apply at 80% or more; Pilot if improvement is likely but uncertain; Hold otherwise
LLM	Turns plain-language what-ifs into scenarios, explains results, writes student offer text	Never produces numbers. The app works without it

Not AI, on purpose: menu-health flags, break-even maths and the student budget filter.

What the Kaggle CSV showed

The attached retail_price.csv has 52 products, 9 categories and 20 months (676 rows). It is e-commerce, not food, so campus-food elasticities still need the student survey.

- Pooled elasticity is about −2.4 (95% interval −3.2 to −1.6). That is the realistic range for synthetic items.
- Per-category estimates are too noisy: 7 of 9 intervals include zero. Use partial pooling and never present per-item elasticities as facts.
- Price effects point the right way on later months, but unit forecasts are poor (74 to 98% error). Claim the response, not exact units.
- A free boosted model predicted demand rising with a 10% price rise in 38% of test rows. The price-decreasing constraint removes that.

- Exclude the customers column, which tracks the sales outcome. Use time-ordered validation and no linear time trend.

Synthetic data spec

The synthetic data is four files, generated by simulating a student population so that combos and price effects emerge naturally.

File	Contents
sales_log	The 9 PS fields (item, price, portion_size_g, cost, qty_sold, combo, combo_items, day_type, discount_pct), plus date and a price_test flag
items	Price, portion, cost and role for 7 mains, 4 add-ons and 3 combos
student_tiers	Tight, mid and flexible shares and daily budgets, calibrated from the student survey
hidden_truth	The true elasticities, never shown to the model and used only for grading

How to generate it. Run a simulated student population for 120 days. Each student has a tier budget, picks a main (or skips) by price, portion and taste within budget, then picks an add-on. Combos discount the pair, so cannibalisation appears naturally. Day types are regular, exam, weekend and event, with footfall around 700, 520, 230 and 860 and different tier mixes. Add an unobserved daily crowd shock. Aim for about 1,300 item-day rows plus about 360 combo rows.

Planted cases. Tune the generator until each case behaves as labelled.

Case	Plant
Popular loss-maker	Chicken Biryani ₹90, cost ₹96, around 70 a day
Mild loss-maker	Paneer Roll ₹55, cost ₹57
Poor sellers	Salad Bowl ₹65 and Fresh Juice ₹45, around 10 a day each
Underpriced star	Dosa ₹40, cost ₹24, high volume

Case	Plant
Combo that works	Veg Meals + Tea
Cannibalisation trap	Dosa + Cold Coffee
Wrong combo	Chicken Biryani + Fresh Juice, which bundles a loss-maker

Price variation, so elasticity is learnable. Use two regimes. For the first 60 days, prices follow the unobserved crowd, which is biased, like real managers. For the next 60 days, run randomised test weeks of ±5, ±10 and ±15%. Show the owner why test weeks matter. Vary combo discounts randomly (0, 5, 10, 15%) by week.

Calibration. Draw typical item elasticities from the real-data range (about −1.6 to −3.2), with the tight tier more price-sensitive. Set tier budgets from the survey.

Slide 1. Say the data is synthetic, drawn from the real-data elasticity range and calibrated on the student survey responses.

How it recommends prices and combos that suit students

The recommender maximises profit only inside guardrails the owner sets, so every student-facing offer is already profit-safe. It works in eight steps:

1. **Diagnose** with rules: loss-makers, poor sellers and stars.
2. **Generate candidates:** price grid within ±15%, portion tiers, and combo discounts.
3. **Predict** each candidate with ranges.
4. **Filter** by the guardrails below.
5. **Optimise:** maximise expected profit subject to the owner's coverage target. Sweep the target to draw the frontier.
6. **Gate** each move as Apply, Pilot or Hold. A Pilot comes with a one-week price-test plan.
7. **Choose combos:** pair a high-margin add-on with a high-traffic main, pick the discount whose break-even take rate is plausible, and reject any combo with negative incremental profit.
8. **Student view:** show only approved offers within the student's budget, ranked by value.

Owner-set guardrails:

- Margin floor.
- Maximum weekly price change.

- Coverage target for student budgets.
- Combo discount cap.
- Fairness mode: no surge pricing on exam or event days.
- No silent shrinking. Portions are visible tiers with ₹ per gram, which serves tight budgets without discounting the regular plate.

The research behind these choices: price rises tied to costs are accepted but demand-driven surges seem unfair (Kahneman, Knetsch and Thaler); downsizing is judged less fair than an equal price rise (Evangelidis); and pricing by bundle size gets close to full mixed bundling with few prices (Chu, Leslie and Sorensen).

What is distinctive

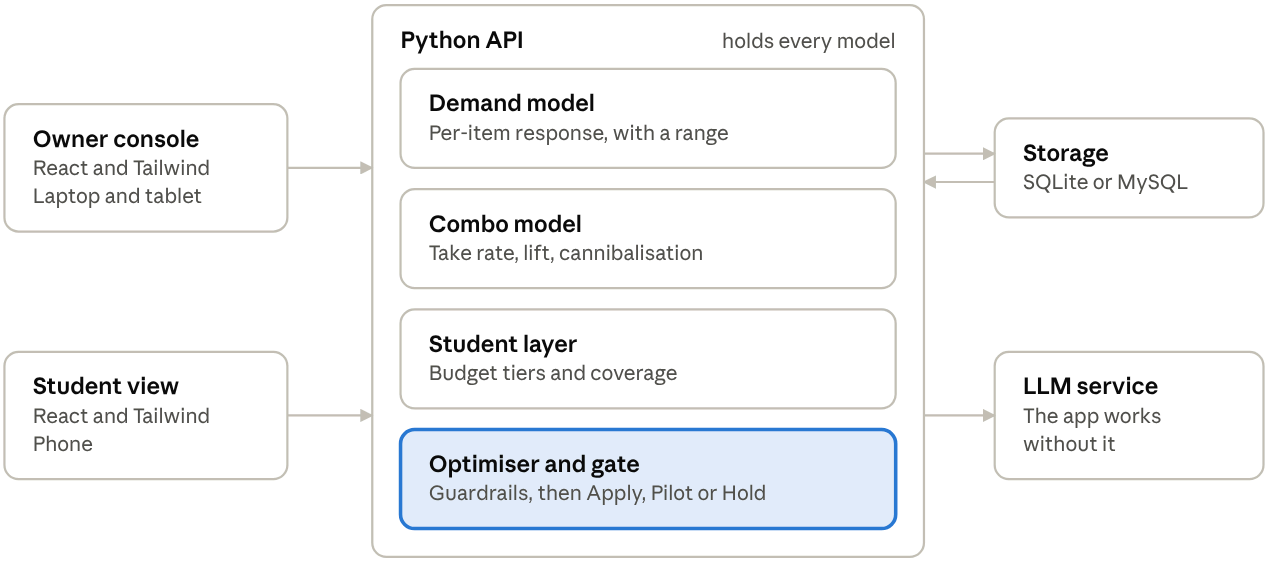
These seven features go beyond the typical forecast-and-chart entry. I have not surveyed other teams, so this is a comparison with that typical entry, not a claim of uniqueness.

1. **Profit-versus-affordability frontier** with a "price of affordability" number.
2. **Apply, Pilot or Hold gate** that recommends an experiment when the data is too thin, turning noisy data into a feature.
3. **Cannibalisation-aware combo engine** that shows combos that lose money.
4. **Portion ladder** instead of silent shrinking.
5. **Student Budget Builder** that can only show profit-safe offers.
6. **Grading against the hidden world:** did the model recover the planted elasticities, and how much of the achievable gain did the recommendation capture? This is an in-world check only; real validation is a price A/B test.
7. **Override log** that feeds the next cycle.

Architecture and build schedule

A Python API holds all the models, and two React front ends, a database and an optional LLM service connect to it. The frontend is React and Tailwind and responsive. Storage is SQLite for speed, or MySQL if you prefer. The LLM is called through an API with a no-internet fallback.

A Python API holds all the models behind two React front ends



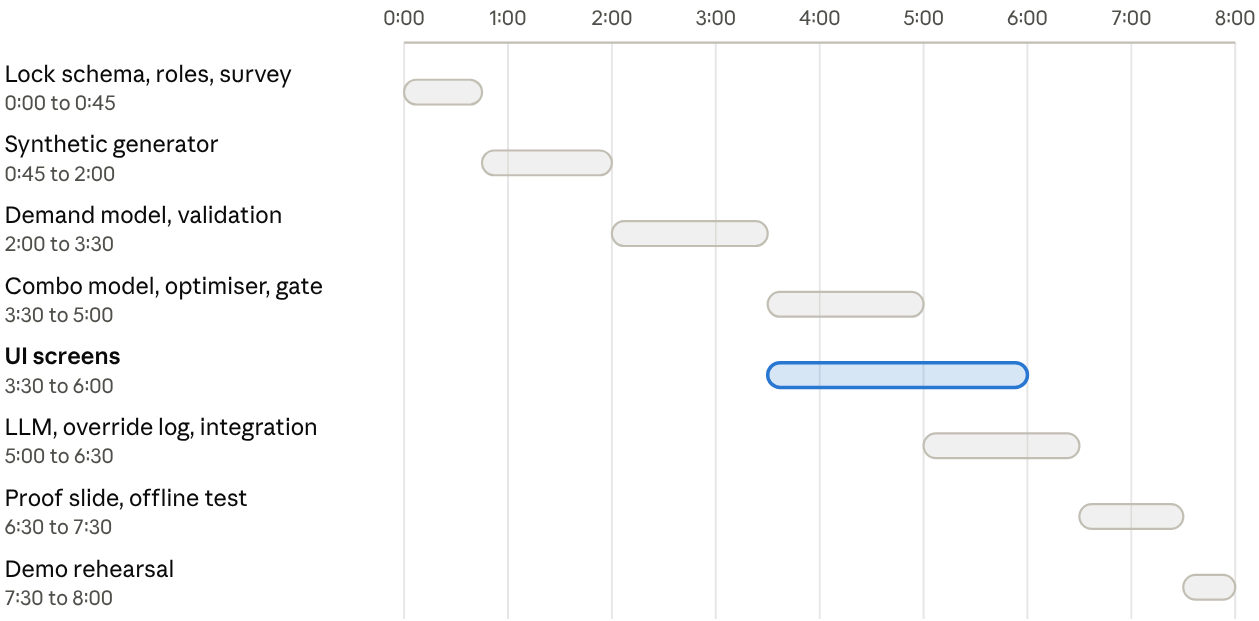
Endpoints: items, predict, compare scenarios, evaluate combo, recommend, student offers, overrides

architecture · 2 front ends, 1 API

The build takes 8 hours with 5 people.

UI screens are built in parallel with the combo model and optimiser

Build window of 8 hours, 5 people



build schedule · 8 hours, 5 people

If time is short, drop the LLM, the boosted challenger and the frontier chart, and keep the spine: generator, model, scenario compare and recommendation.

Demo and proof

The demo runs five minutes in this order: menu health, Demand Lab, scenario comparison, the combo trap, the frontier, then the student view.

Proof to report:

- Demand error on time-ordered data.
- Whether the planted elasticities were recovered.
- Profit gain against today's prices, with a range.
- Students served within budget.
- Guardrail breaches (target: zero).

Likely judge questions:

- *Is synthetic data valid?* It tests the pipeline. A price A/B test is the real check, and the student survey calibrates the parameters.
- *Why not let an LLM price?* It cannot be audited, so the LLM only explains computed numbers.
- *Could this hurt students?* Hard caps, limits on weekly change and visible portions prevent that.

Limits and sources

Three limits apply to this plan:

- The Kaggle data is e-commerce, so campus-food elasticities need the student survey.
- The literature points are from search excerpts, not full papers, so check each before it goes on a slide.
- I found no TCS judging rubric.

Sources:

- Retail Price Optimization dataset (Kaggle), as named in the PS; the numbers above come from the attached retail_price.csv.
- Kahneman, Knetsch and Thaler, Fairness as a Constraint on Profit Seeking (1986)
- Evangelidis, Shrinkflation Aversion (Marketing Science, 2024)

- Chu, Leslie and Sorensen, Bundle-Size Pricing as an Approximation to Mixed Bundling